SOFTWARE ENGINEERING - ASSIGNMENT 2

CLASSSYNC AI: AUTOMATED UNIVERSITY TIMETABLING SYSTEM

Group Members:
-

-

-

Section: Software Engineering - Section C
Date of Submission: December 4, 2025

1. FUNCTIONAL REQUIREMENTS

1.1 Input Management

FR1.1.1: The system shall allow users to upload course information via CSV file containing course code, course name, instructor name, section identifier, credit hours, and required room type.

FR1.1.2: The system shall allow users to upload room information via CSV file containing room number, room type (lecture hall/laboratory/tutorial room), and seating capacity.

FR1.1.3: The system shall validate uploaded CSV files for correct file structure, presence of all required fields, and appropriate data types for each field.

FR1.1.4: The system shall display specific error messages indicating the exact nature of validation failures including missing fields, invalid data types, incorrect formatting, or structural issues.

FR1.1.5: The system shall parse validated CSV files into internal data models for processing by the scheduling engine.

FR1.1.6: The system shall reject invalid files and prevent further processing until validation errors are corrected.

FR1.1.7: The system shall maintain a log of all file upload attempts including timestamps, filenames, and validation results.

1.2 Scheduling Engine

FR1.2.1: The system shall generate conflict-free timetables using genetic algorithm optimization that evolves solutions over multiple generations.

FR1.2.2: The system shall prevent instructor double-booking by ensuring no instructor is assigned to multiple courses in the same time slot.

FR1.2.3: The system shall prevent room double-booking by ensuring no room is assigned to multiple courses in the same time slot.

FR1.2.4: The system shall prevent section conflicts by ensuring no section has multiple courses scheduled simultaneously.

FR1.2.5: The system shall match each course to an appropriate room type based on course requirements (lecture requires lecture hall, lab requires laboratory).

FR1.2.6: The system shall ensure assigned room capacity meets or exceeds the expected section enrollment for each course.

FR1.2.7: The system shall assign courses only to valid time slots within university operating hours (8:00 AM to 6:00 PM, Monday to Friday).

FR1.2.8: The system shall automatically repair invalid schedule assignments through intelligent rescheduling when constraint violations are detected.

FR1.2.9: The system shall continue optimization until either the maximum number of generations is reached or solution fitness converges.

FR1.2.10: The system shall maintain the best solution found across all generations regardless of subsequent population changes.

1.3 Constraint Management

FR1.3.1: The system shall enforce hard constraints that must never be violated including room type compatibility, room capacity sufficiency, time slot validity, and elimination of all scheduling conflicts.

FR1.3.2: The system shall optimize soft constraints that improve schedule quality including balanced instructor workload distribution, scheduling consecutive time slots for multi-hour lab sessions, and minimizing idle gaps in daily schedules.

FR1.3.3: The system shall allow administrators to adjust relative importance weights for each soft constraint before initiating schedule generation.

FR1.3.4: The system shall provide default constraint weights based on common institutional priorities that can be customized as needed.

FR1.3.5: The system shall always prioritize satisfaction of hard constraints over optimization of soft constraints in all scheduling decisions.

FR1.3.6: The system shall calculate and display constraint satisfaction rates for both hard and soft constraints in the final solution.

1.4 Output Generation

FR1.4.1: The system shall generate section-wise timetables displaying all courses assigned to each academic section organized by day and time slot.

FR1.4.2: The system shall generate instructor-wise timetables displaying the complete teaching schedule for each instructor organized by day and time slot.

FR1.4.3: The system shall generate room-wise timetables displaying utilization schedules for each room organized by day and time slot.

FR1.4.4: The system shall export all timetables to Microsoft Excel format with separate worksheets for section view, instructor view, and room view.

FR1.4.5: The system shall apply color-coding to timetable cells to distinguish between different course types (lectures, labs, tutorials) and highlight any remaining constraint violations.

FR1.4.6: The system shall include course details in each timetable cell including course code, course name, instructor, room number, and section identifier.

FR1.4.7: The system shall generate a comprehensive performance summary report displaying final fitness score, hard constraint satisfaction rate, soft constraint satisfaction rate, overall room utilization percentage, and generation count.

FR1.4.8: The system shall format exported Excel files with appropriate column widths, cell borders, and header formatting for professional presentation.

1.5 User Interface

FR1.5.1: The system shall provide a graphical user interface accessible to all authorized users for performing scheduling operations.

FR1.5.2: The system shall display file upload status with visual progress indicators during file processing.

FR1.5.3: The system shall allow users to configure genetic algorithm parameters including population size, crossover rate, mutation rate, and maximum generation limit.

FR1.5.4: The system shall display real-time progress updates during schedule generation showing current generation number, best fitness score achieved, and estimated time remaining.

FR1.5.5: The system shall provide immediate visual feedback for successful operations using confirmation messages.

FR1.5.6: The system shall display clear, actionable error messages when operations fail, indicating the cause and suggesting corrective actions.

FR1.5.7: The system shall allow users to save and load configuration profiles for repeated use of preferred parameter settings.

FR1.5.8: The system shall provide a dashboard view summarizing system status, recent activities, and quick access to primary functions.

2. SYSTEM ACTORS

2.1 Human Actors

2.1.1 Timetable Coordinator (Primary User)

The principal system user responsible for managing the complete timetabling workflow. This role uploads course and room data files, validates input data, configures scheduling parameters and constraint weights, initiates the optimization process, monitors generation progress, reviews generated timetables, and exports final schedules. The Timetable Coordinator has comprehensive access to all system features and serves as the primary interface between institutional requirements and system operations.

2.1.2 Administrator

The technical user responsible for system configuration, maintenance, and troubleshooting. This role manages genetic algorithm parameter defaults, sets institutional constraint policies, tunes performance settings, monitors system logs for errors or issues, and resolves technical problems that arise during operation. Administrators ensure the system operates efficiently and meets institutional technical requirements.

2.1.3 Instructor

An indirect stakeholder who receives their personal teaching schedule generated by the system. Instructors review their assigned courses and time slots, report scheduling conflicts or availability issues to the Timetable Coordinator, and may communicate preferences regarding teaching times or course distribution. While instructors do not directly interact with the system interface, their constraints and feedback significantly influence scheduling decisions.

2.1.4 University Management

Executive-level stakeholders who oversee the timetabling process and make policy decisions. This group reviews timetable quality metrics and performance reports, approves final schedules before publication, establishes institutional scheduling policies and priorities, and makes decisions on resource allocation including room assignments and instructor workloads. Management uses system outputs for strategic planning and resource optimization.

2.2 System Actors

2.2.1 Genetic Algorithm Engine

The core computational component that performs schedule optimization through evolutionary algorithms. This internal actor generates initial random schedule populations, evaluates candidate schedules against all constraints, calculates fitness scores for each solution, applies genetic operators including selection, crossover, and mutation, manages population evolution across generations, and identifies the optimal schedule from all evaluated candidates.

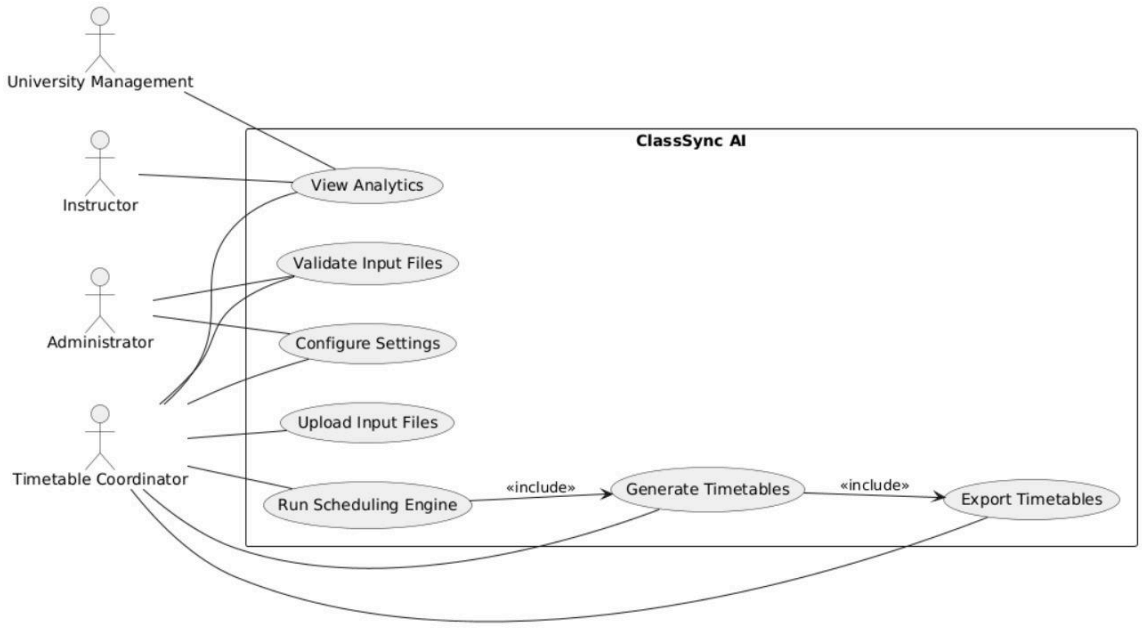
2.2.2 Data Validation Module

An internal component responsible for ensuring data integrity before processing. This actor verifies CSV file structure and format compliance, checks for presence of all required data fields, validates data types and value ranges, identifies missing or inconsistent data, generates detailed validation error reports, and prevents processing of invalid data that could compromise schedule quality.

2.2.3 Output Generation Module

The component that transforms optimized schedule data into user-friendly formats. This actor converts internal schedule representations into readable timetable views, organizes schedules by section, instructor, and room perspectives, applies formatting and color-coding for visual clarity, exports timetables to Microsoft Excel format, and generates performance summary reports with statistical analysis of the scheduling solution.

3. USE CASE MODEL



Use Cases (ovals inside system boundary "ClassSync AI"):

1. Upload Input Files
2. Validate Input Files
3. Configure Settings
4. Run Scheduling Engine
5. Generate Timetables
6. Export Timetables

Relationships:

• Timetable Coordinator → all six use cases
• Administrator → Upload Input Files, Configure Settings
• Upload Input Files <<include>> Validate Input Files
• Run Scheduling Engine → Genetic Algorithm Engine
• Validate Input Files → Data Validation Module
• Generate & Export Timetables → Output Generation Module

4. USE CASE DESCRIPTIONS

Use Case 1: Upload Input Files

Primary Actor: Timetable Coordinator

Supporting Actor: Administrator, Data Validation Module

Description:
The Timetable Coordinator uploads course information and room availability data to the system in CSV format. These files contain all necessary information for schedule generation including course details, instructor assignments, room specifications, and capacity constraints.

Preconditions:
- The system is running and accessible
- The Timetable Coordinator has valid login credentials
- Course and room data files are prepared in correct CSV format
- The system has sufficient storage space for file upload

Postconditions:
- Course and room CSV files are successfully stored in the system database
- Files are marked as "uploaded" and queued for validation
- Upload timestamps and file metadata are recorded in system logs
- The "Validate Files" function becomes enabled for user access

Normal Flow:
1. The Timetable Coordinator logs into the ClassSync AI system.
2. The Coordinator navigates to the "Data Management" section.
3. The Coordinator selects "Upload Input Files."
4. The system displays a file upload interface with two upload areas.
5. The system prompts the Coordinator to select the course information CSV file.
6. The Coordinator browses and selects the course data file.
7. The system uploads the file and displays progress.
8. The system confirms successful upload with a checkmark.

9. The system prompts for the room information CSV file.
10. The Coordinator selects the room data file.
11. The system uploads and confirms the room file.
12. The system displays a summary with both filenames, sizes, and upload times.
13. The system enables the "Validate Files" button.

Exception Flow: File Upload Failure

E1: During upload, the network connection is interrupted.
E2: The system detects the upload failure.
E3: The system displays: "Upload failed due to connection error. Please try again."
E4: The system discards partial data and returns to file selection.

Use Case 2: Validate Input Files

Primary Actor: Timetable Coordinator

Supporting Actor: Data Validation Module

Description:

The system performs comprehensive validation of uploaded CSV files to ensure data integrity, completeness, and correct structure. The validation process checks for required fields, appropriate data types, logical consistency, and identifies any errors that would prevent successful schedule generation.

Preconditions:

- Course and room CSV files have been successfully uploaded
- Files are stored and accessible in the system database
- The Data Validation Module is operational

Postconditions:

- All files are marked as either "valid" or "invalid"
- Detailed validation reports are generated
- Valid files are flagged as ready for processing
- Invalid files are marked for correction with error details logged

Normal Flow:

1. The Coordinator selects "Validate Files."
2. The system initiates the Data Validation Module.
3. The module checks course file for required columns (Course Code, Name, Instructor, Section, Credit Hours, Room Type).
4. The module verifies all rows contain values with no empty cells.
5. The module validates data types (Credit Hours must be numeric).
6. The module checks for duplicate course-section combinations.
7. The system validates the room file similarly.
8. The system compiles validation results into a report.
9. The system displays total records, valid count, invalid count, and error details.
10. If valid, the system enables "Configure Settings" and "Run Scheduling Engine."
11. The system displays: "Validation complete. All files are valid."

Exception Flow: Missing Required Headers

E1: The module detects missing required column headers.
E2: The system stops validation immediately.
E3: The system displays: "Validation failed. Required column 'X' is missing."
E4: The system marks the file as "Invalid" and disables "Run Scheduling Engine."

Use Case 3: Configure Settings

Primary Actor: Timetable Coordinator

Supporting Actor: Administrator

Description:
The user configures genetic algorithm parameters and adjusts constraint importance weights to customize the scheduling optimization process according to institutional priorities.

Preconditions:
- The system is running and user is authenticated
- User has appropriate permissions to modify settings
- Default configuration values are loaded

Postconditions:
- Updated parameters are saved in configuration database
- New constraint weights will be applied in subsequent runs
- Configuration changes are logged with timestamp

Normal Flow:
1. The Coordinator selects "Configure Settings."
2. The system displays the Settings panel with two tabs: "Algorithm Parameters" and "Constraint Weights."
3. The Coordinator selects "Algorithm Parameters."
4. The system displays: Population Size, Crossover Rate, Mutation Rate, Maximum Generations.
5. The Coordinator modifies Population Size.
6. The system validates the input is within range (50-1000).
7. The Coordinator selects "Constraint Weights" tab.
8. The system displays sliders for: Balanced Workload, Consecutive Labs, Minimize Gaps, Preferred Times.
9. The Coordinator adjusts weights using sliders.
10. The Coordinator selects "Save Configuration."
11. The system validates all values are within acceptable ranges.
12. The system displays: "Save these configuration changes?"
13. The Coordinator confirms.
14. The system saves parameters to database.
15. The system displays: "Configuration saved successfully."

Exception Flow: Parameter Out of Range
E1: The Coordinator enters a value outside acceptable range.
E2: The system highlights the field in red.
E3: The system displays: "Value must be between 50 and 1000."
E4: The system disables "Save" until corrected.

Use Case 4: Run Scheduling Engine

Primary Actor: Timetable Coordinator

Supporting Actor: Genetic Algorithm Engine, Data Validation Module

Description:
The Coordinator initiates automated scheduling where the system generates a conflict-free timetable using genetic algorithm techniques. The system creates multiple candidate schedules, evaluates them against constraints, and evolves them over generations to discover the optimal solution.

Preconditions:
- CSV files have been uploaded and validated
- All required data fields are present and correct
- Scheduling parameters have been configured
- Constraint weights have been set
- Genetic Algorithm Engine is initialized

Postconditions:
- An optimized timetable solution is generated and stored
- Fitness score and performance metrics are calculated
- Solution satisfies all hard constraints
- Solution is ready for timetable generation and export
- Complete scheduling session is logged

Normal Flow:

1. The Coordinator selects "Run Scheduling Engine."
2. The system verifies all preconditions are met.
3. The system displays a summary: number of courses, rooms, time slots, and parameters.
4. The system displays estimated completion time.
5. The Coordinator selects "Start Scheduling."
6. The system initializes the Genetic Algorithm Engine.
7. The system creates an initial population of random candidate schedules.
8. The system displays progress panel: Current generation, Best fitness, Progress bar, Time remaining.
9. For each candidate, the system evaluates hard constraints (no conflicts, room match, capacity).
10. The system evaluates soft constraints (workload balance, lab continuity, gap minimization).
11. The system calculates composite fitness scores (0-100 scale).
12. The system identifies the best schedule from initial population.
13. The system enters evolutionary loop: selection, crossover, mutation.
14. The system applies repair mechanisms to fix constraint violations.
15. The system updates display after each generation.
16. The system continues until maximum generations reached.
17. The system finalizes the best schedule found.
18. The system calculates metrics: fitness, constraint satisfaction, room utilization, workload stats.
19. The system stores the optimized schedule.
20. The system displays completion dialog with final metrics.
21. The system enables "Generate Timetables" button.
22. The system logs the complete session.

Exception Flow: No Feasible Solution Found

E1: After all generations, no schedule satisfies minimum feasibility criteria.
E2: The system identifies which hard constraints could not be satisfied.
E3: The system displays: "Unable to generate feasible schedule. Conflicts: Insufficient labs, Instructor overbooked."
E4: The system suggests: "Consider: Adding rooms, reducing courses, adjusting hours."
E5: No schedule is saved.

Use Case 5: Generate Timetables

Primary Actor: Timetable Coordinator

Supporting Actor: Output Generation Module

Description:
The system transforms optimized schedule data into three comprehensive timetable views organized by section, instructor, and room. Each view presents the same scheduling information from different perspectives tailored to specific stakeholder needs.

Preconditions:
- Scheduling engine has successfully completed
- A valid conflict-free schedule exists in database
- Output Generation Module is operational
- All course, instructor, and room data is available

Postconditions:
- Section-wise timetables are generated
- Instructor-wise timetables are generated
- Room-wise timetables are generated
- All timetables are stored in memory ready for export
- Generation success is logged

Normal Flow:
1. The Coordinator selects "Generate Timetables."
2. The system displays: "Generate timetables from optimized schedule?"
3. The Coordinator confirms.
4. The system displays progress: "Generating timetables..."

5. The system retrieves optimized schedule data.
6. The system creates section-wise view: identifies sections, creates grids, populates cells.
7. The system updates progress: "Section timetables: Complete (33%)"
8. The system creates instructor-wise view: identifies instructors, creates grids, calculates teaching hours.
9. The system updates progress: "Instructor timetables: Complete (66%)"
10. The system creates room-wise view: identifies rooms, creates grids, calculates utilization.
11. The system updates progress: "Room timetables: Complete (100%)"
12. The system validates all timetables for consistency.
13. The system stores all views in memory.
14. The system displays: "Timetable generation successful. Ready for export."
15. The system shows summary: count of each timetable type, average room utilization.
16. The system enables "Export Timetables" button.

Exception Flow: Schedule Data Corrupted
E1: The system cannot access schedule data.
E2: The system displays: "Cannot access schedule data. May be corrupted."
E3: The system suggests: "Please re-run the scheduling engine."
E4: The system returns to main menu.

Use Case 6: Export Timetables

Primary Actor: Timetable Coordinator

Supporting Actor: Output Generation Module

Description:
The user exports generated timetables to Microsoft Excel format for distribution and

printing. The system creates professionally formatted Excel files with separate worksheets, applies color-coding, and saves to a specified location.

Preconditions:

- Timetables have been successfully generated
- Output Generation Module is operational
- System has write access to output directory
- Excel file libraries are available

Postconditions:

- Excel files are saved to specified location
- Files are formatted with color-coding, borders, headers
- Performance summary is included
- Export activity is logged

Normal Flow:

1. The Coordinator selects "Export Timetables."
2. The system displays export dialog with default output location and file naming options.
3. The Coordinator reviews settings.
4. The Coordinator selects "Export."
5. The system creates Excel workbook: "Timetable_[date]_[time].xlsx"
6. The system creates "Section Timetables" worksheet with formatted headers and grids.
7. The system populates section data with course details.
8. The system applies color-coding: Lectures=blue, Labs=green, Tutorials=yellow.
9. The system creates "Instructor Timetables" worksheet.
10. The system creates "Room Timetables" worksheet.
11. The system creates "Performance Summary" worksheet with metrics.
12. The system sets print areas and page breaks.
13. The system saves workbook to output folder.
14. The system verifies file written successfully.
15. The system displays: "Timetables exported successfully! File: [path]"
16. The system offers: "Open File," "Open Folder," "Close."
17. The Coordinator selects "Open File."

18. The system launches Excel with the file.
19. The system logs export activity.

Exception Flow: Output Folder Inaccessible

E1: The system cannot write to folder due to permissions.
E2: The system displays: "Cannot save file. Folder is inaccessible or write-protected."
E3: The system shows folder browser: "Please select alternate location."
E4: The Coordinator selects new accessible folder.
E5: The system attempts save to new location.

5. SYSTEM OVERVIEW

ClassSync AI is an intelligent university timetabling system that automates the complex and time-intensive process of academic schedule generation through the application of genetic algorithm optimization techniques. The system addresses the multifaceted challenge of assigning hundreds of courses to appropriate time slots and suitable rooms while simultaneously satisfying numerous institutional constraints, instructor requirements, and educational quality standards. Users initiate the process by uploading comprehensive course information and room availability data in standardized CSV format, which the system rigorously validates to ensure data completeness, structural correctness, and logical consistency before proceeding with optimization. The Timetable Coordinator then configures essential scheduling parameters including genetic algorithm settings (population size, crossover rate, mutation rate, generation limits) and adjusts constraint importance weights according to specific institutional priorities and policies. The sophisticated genetic algorithm engine generates an initial population of candidate schedules and systematically evolves them through multiple generations using selection, crossover, and mutation operators, automatically preventing all forms of scheduling conflicts including instructor double-booking, room allocation overlaps, and section time clashes while simultaneously optimizing for balanced faculty workloads, efficient room utilization, and minimized gaps in student schedules. Once the optimization process converges to an optimal or near-optimal solution, the system produces comprehensive, professionally formatted timetables organized from three distinct perspectives: section-wise schedules for student planning, instructor-wise schedules for faculty coordination, and room-wise schedules for facility

management. These timetables are exported as color-coded Microsoft Excel spreadsheets featuring clear formatting, visual distinction between course types, and embedded performance analytics including fitness scores, constraint satisfaction rates, and resource utilization statistics. By completely eliminating the manual scheduling burden and human error inherent in traditional timetabling approaches, ClassSync AI dramatically reduces schedule creation time from several weeks of intensive labor to mere minutes of automated processing while simultaneously ensuring superior quality outcomes characterized by zero scheduling conflicts, optimized resource allocation, and improved satisfaction of both hard institutional requirements and soft preferential objectives that enhance overall academic experience and operational efficiency.